Software Design File

SQRT (Sub-zero Quantitative Research Team)

Bruce Ritter, Elyana Abdallah, Javier Ayuso, Orlaith Ni Dhuill, Caimin Keavney

Document History

Date	Version	Notes	Approver
2026–02–25	SQT-CDR-SDF-0	Initial release	CK

Author Contributions

BR §1–6

EA §6

JA §5

OND §2

CK §2–5, Reviewer

Contents

1 Executive Summary **3**

2 Mission Overview **4**
2.1 Mission Statement & Goals 4
2.2 Software Tasks 4

3 Software Design Overview **5**
3.1 Software Behavior Overview 5
3.2 Design Standard and Conventions 6

4 Software Architecture **7**
4.1 Software Product Tree 7
4.2 Software Block Diagram 7

5 Software Interface Requirements **10**

6 Software Development & Verification Plan **12**

List of Figures

1 High Level Software Architecture 5
2 Software Product Tree 7
3 Software Flow Diagram 7
4 Triggering Algorithm Flowchart 9

List of Tables

1 Software Interfaces 10
2 External (Component) Interfaces 11
3 Software Development and Verification Tasks 12

Key Reference Documents

1. SQT-CDR-PMP-0
2. SQT-CDR-DMP-0
3. SQT-CDR-SEP-0

1 Executive Summary

This file provides an overview of the software design for the SQRT mission. SQRT will investigate the thermal evolution of water undergoing flash evaporation by exposing water to a low pressure environment during a high altitude balloon flight. Environmental and location data must be collected to satisfy mission requirements, and the water exposure valve must be opened when triggered precisely at the target pressure range. This document details the development plan and organization of the mission software product, which is derived from key operations required to fulfil mission requirements.

Section 2 provides an overview of the mission requirements that drive necessary software tasks. The primary responsibilities of the software are outlined in Section 3. The key operations necessary to complete all software tasks are detailed. Section 4 formats the responsibilities in a software architecture overview showing the flow of operations. The interfaces between components and software functions are detailing in Section 5. The software development plan in Section 6 will facilitate the full completion of all software tasks and associated work packages.

2 Mission Overview

2.1 Mission Statement & Goals

Mission Statement: The SQRT mission will produce and observe the flash evaporation of water via exposure to a low pressure and low temperature environment to investigate how such plumes thermally evolve.

The flash evaporation process is relevant to cryovolcanic plumes on icy moons in our solar system as well as desalinisation plants on Earth. In order to reproduce the conditions required for flash evaporation, this investigation will be done using a high-altitude demonstration. An isolated water container on board will be flown to an altitudes in excess of 20 km, where temperatures and pressures are lower. Upon sudden exposure to this environment, it is expected that the water will undergo a near-instantaneous phase transition. SQRT software will facilitate observing this process via data collection, processing, storage, and transmission to ground.

2.2 Software Tasks

The software will be designed to run the Raspberry Pi onboard computer which will interface with the following components: DS18B20 thermometer used to collect internal tuppersat temperatures, MS5611-01BA03 pressure sensor to collect ambient pressure, RF Solutions ANT-GPS-P20-SMA GPS to collect position data, Adafruit MLX90640 thermal sensor to collect a thermal "image" of the low pressure chamber, Tuppersat Telemetry Thingamabob antenna for communications, and DFROBOT SER0043 servo motor to actuate the valve system. An SD card will interface with the Pico for local storage.

Our project will produce one software product to drive the onboard computer and operate the various components for the purpose of meeting all mission goals (outlined in SQT-CDR-SEP-0). The software will need to autonomously read and record data from sensors routinely in a loop. A triggering algorithm will also be implemented within the loop to check for the conditions to expose the water to the low pressure chamber. It is critical that the water sample be exposed to the low pressure chamber at the most optimal environmental conditions during flight to provide the best opportunity to observe flash evaporation. The software will also handle calibration, storage, and transmission of data.

3 Software Design Overview

The SQRT mission software will be designed to behave with the structure and key operations outlined in this section to ensure necessary data is collected, processed, stored, and transmitted correctly. Team members will follow the design standards and conventions outlined in this section to ensure that code written by different authors can be easily adapted and implemented together.

3.1 Software Behavior Overview

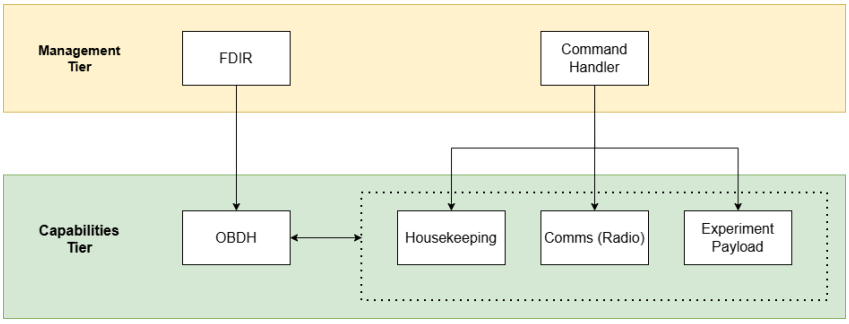


Figure 1: High Level Software Architecture

At a high level, the SQRT software can be divided into two tiers: Management and Capabilities, as shown in Figure 1. The Management tier encompasses how the onboard computer sends commands to collect, save, process, and transmit data. It additionally encompasses code handling fault detection, isolation, and recovery (FDIR) in case components fail. The capabilities tier breaks down the commands and FDIR into sections of operations for onboard data handling (OBDH), platform needs (housekeeping data handling), payload needs (science data handling and triggering experiment processes), and communications. OBDH will interact with all sections stemming from the Command Handler. A more detailed description of the software architecture can be found in Section 4. This section provides an overview of the software design including high-level operations and design standards that the SQRT team will follow to develop effective software.

To satisfy the mission requirements, the payload will constantly collect data from housekeeping sensors and experiment specific sensors within a sectioned data collection loop. A triggering algorithm will be used during each loop to check for the target conditions for water exposure. If a targeting condition is met, the loop shall be interrupted and the valve will be opened via servo motor activation. Data processing (calibration, formatting, and storage) shall take place directly after sensor reading for each sensor. Packets of science and telemetry data shall be transmitted to the ground consistently. SQRT software responsibilities are driven by the mission requirements and organized into key operations below.

Key Operations:

1. **Command Control:** Control the looped data collection, processing, and data packet transmission and interrupt everything if trigger conditions are met to run the valve opening operations before returning to the loop. Handle the different rates of collection and transmission within said loop.
2. **Housekeeping Data Collection Operations:** Collect data from the GPS, internal temperature sensor (thermometer), and pressure sensor.
3. **Triggering Payload Operations:** Use a triggering algorithm to open the valve if: the payload is at a target pressure, if the pressure values are strange but the payload hits a target altitude, or if the payload begins its decent before the former two conditions are met. A flowchart of this operation is shown in Figure 4.

4. **Thermal Sensor Data Collection & Processing:** Collect and calibrate thermal sensor data. Calibration functions will be adapted from a public Adafruit micropython library for this component [1]. A Trade-off analysis will be preformed during software development to determine if processing will be done after each frame is taken by the thermal sensor or if arrays and calibration values will be stored for processing in a seperate block of time. This will be determined based on an anlysis of computing time.
5. **Storing Data:** Store calibrated data in structured files on the onboard SD card.
6. **Creating Data Packets:** Create data packets for transmission: 2 packets for housekeeping/telemetry data, and one packet for science data.
7. **Data Packet Transmission:** Transmit data packets to ground. The rate of transmission will differ from the rate of data collection, as shown in the software flow. Further information on data transmission and management is included in SQT-CDR-DMP-0.

3.2 Design Standard and Conventions

All software developed for the SQRT project will be stored and updated within the SQRT GitHub managed by the software lead. Team members will commit changes to the code in their own named branches to protect code from being overwritten, and commits to the main branch will be added using requests rather than automatically pushing for the same reason.

The GitHub will be structured with folders for components which will include code, test data, and other relevant files. All folders will include a README.md with information on version history, key functions/files, authors, and use cases that will be kept up to date.

Software product modules will be written in Python, and the structure of the code will follow PEP8 style guidelines [2]. Code style standards include using docstrings to all functions with a clear description of inputs and outputs as well as notes on implementation.

Functions for operating specific sensors will be organised in classes. Classes developed for the project will include SQT in their name, and other naming conventions will follow PEP8 style guidelines and be relevant to the code function.

Our design methodology is to work from a higher level (key operations that must be carried out by the software) down to finer details (individual functions). The software design will prioritise checks and redundancies since the payload must run autonomously. During design, clear documentation will also be prioritised within the GitHub to ensure all changes to software are kept track of by the team and a code history is available if needed.

4 Software Architecture

SQRT software will run the onboard computer autonomously for the duration of the flight. The software is designed to run for at least 6 hours, per system requirements outlined in SQT-CDR-SEP-0. All operations will run as one software product running the Raspberry Pi Pico onboard computer. The key operations outlined in Section 3 will run within a timed loop based on the timing of data collection, analysis, possible valve activation, calibration and compression of data, and transmission.

4.1 Software Product Tree

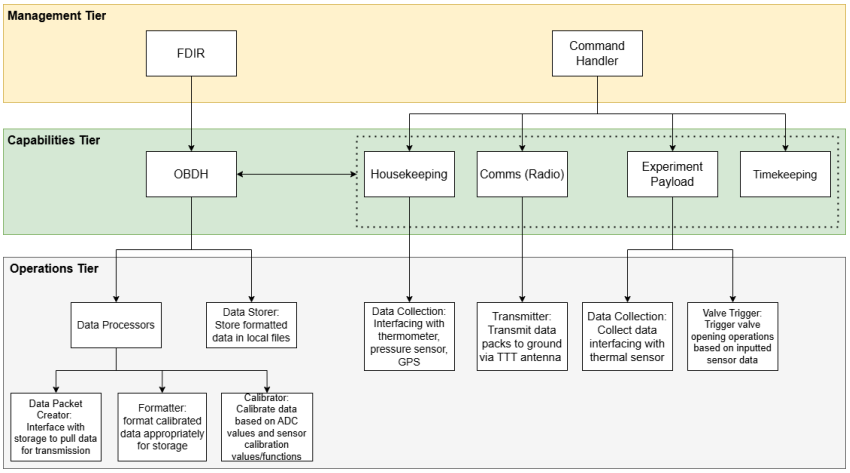


Figure 2: Software Product Tree

The SQRT software product that will run onboard operations is broken down into a hierarchy of modules in Figure 2 above based on the necessary key operations for the mission.

4.2 Software Block Diagram

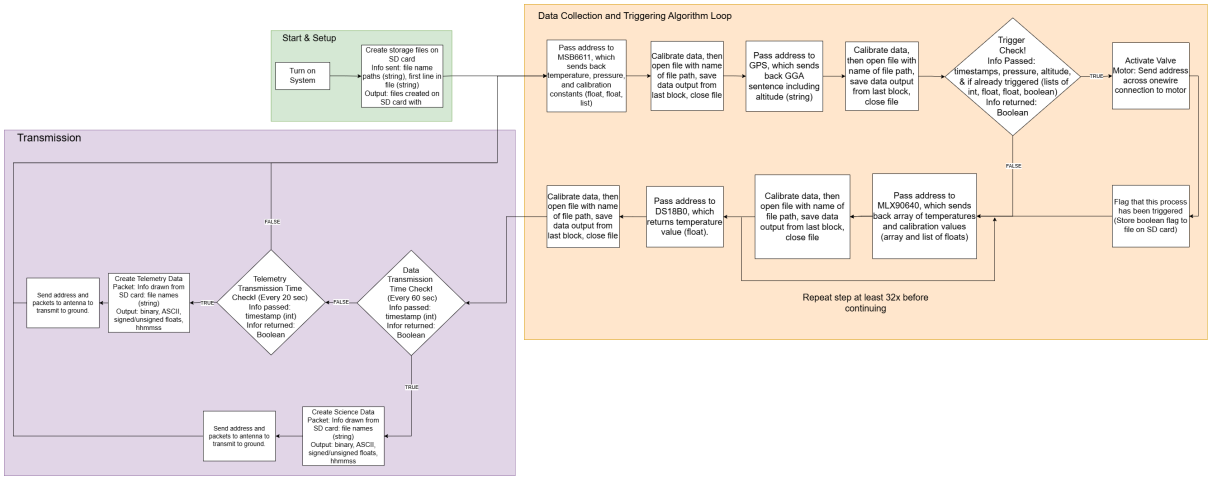


Figure 3: Software Flow Diagram

The SQRT software product will run data collection/processing, triggering checks, and data transmission sequentially. The software flow is shown in Figure 3 above. Data types passed between interfaces are listed with each operation in the flowchart.

As noted, the MLX90640 thermal sensor will be read from at least 32 times within the data collection loop to meet the frames per second requirement as outlined SQT-CDR-SEP-0. All other sensors will be read from once per loop, meeting system requirements outlined in the same document. The trigger algorithm design is described in further detail in Section 4.2. All data will be timestamped and formatted following the design standards and conventions in Section 3.

4.2.1 Software Flow Description

Startup: Files will be created and written to the SD card to hold collected data. The data collection loop (shown in orange) will run immediately after setup tasks are completed and will continue running until the onboard computer is recovered and safely shut down with two deviations:

1. **Transmission:** After each completion of the data collection loop (orange block), the transmission loop (purple block) will run if it is the right time to transmit data ($\geq$ 50 seconds since last transmission) or telemetry ($\geq$ 20 seconds since last transmission). Data transmission is prioritised over telemetry transmission in the loop. Checking for The data collection loop will continue following completion of the transmission tasks.
2. **Valve Trigger:** The valve triggering algorithm ("Trigger Check" in Figure 3) will run during the data collection loop. The data collection loop will be interrupted if the trigger check returns True to expose the water sample.

No trigger condition: If none of the triggering conditions are met, the data collection loop will continue as normal. Data is taken next with the thermal array, which will be run throughout the flight even when water has not yet been released in order to collect background data. Data will be calibrated and stored after data is collected for each sensor reading.

When trigger conditions are met: The data collection loop will be interrupted and the servo motor actuating the valve will be activated. The valve trigger flag will be set to True. The thermal sensor data collection and storage will then run as before. The thermometer will then collect and store data as before, and the loop will continue.

After trigger conditions have been met: After the water release has been triggered, the following data collection loops will skip the triggering algorithm steps as they are no longer needed (triggering algorithm only runs if the trigger flag is False). Data collection, processing, and storage will continue in the same order for all components. The temperature threshold checks from the thermometer located at the water tube exit point will continue. Storage of thermal sensor arrays and the arrays chosen for transmission will differ so that the most scientifically valuable arrays are transmitted to ground during the limited flight time. Storage and transmission is detailed further in SQT-CDR-DMP-0.

4.2.2 Triggering Algorithm Flow

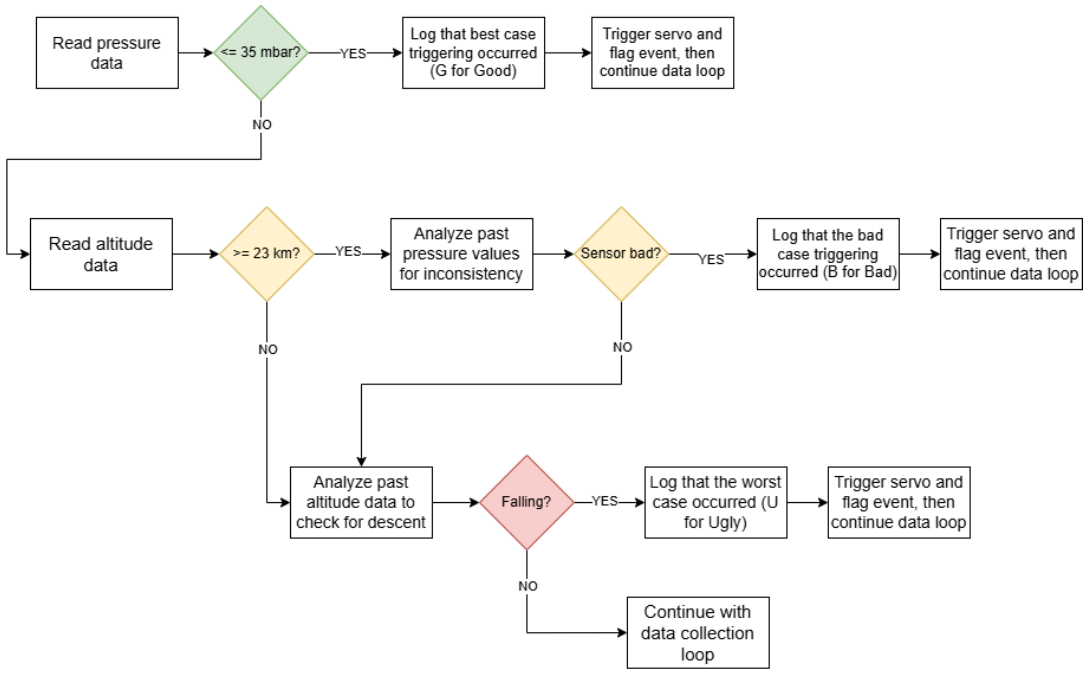


Figure 4: Triggering Algorithm Flowchart

The SQRT mission depends on successfully exposing the onboard water sample to a low pressure environment, where the atmospheric pressure is at or below 35 millibar. The altitude at that pressure will vary around 23 km depending on weather conditions. A triggering algorithm will be developed to ensure water is only exposed at specific conditions.

The flowchart in Figure 4 shows the sequence of checks the triggering algorithm will run through, starting with reading pressure data. If any check returns True, then the algorithm will immediately open the valve and set the trigger flag to True for use with other functions before the data collection loop continues. The type of trigger event is also logged to be transmitted. If all three checks are false, then the data collection loop continues. The triggering algorithm is nested within the data collection loop as shown in Figure 3. The valve used to expose the water sample will need to be opened only if one or more of the following conditions are met:

Good Case (Green): The valve will trigger if ambient atmospheric pressure is $\leq$ 35 millibar.

Bad Case (Yellow): In case the pressure sensors onboard are not functioning or not processed correctly, the software will rely on altitude measurements from the GPS. The valve will trigger if the payload is at an altitude $\geq$ 23 km.

Ugly Case (Red): This check is in place in case the balloon pops prematurely due to payload weight or weather. In this case pressure and altitude targets were not hit since the balloon did not get ascend high enough, and the payload is now descending. The valve will be triggered immediately if descent is detected and the balloon is over 2 km altitude (so that this does not happen while the balloon and payload are being prepared pre-launch).

5 Software Interface Requirements

This section shows internal and external interfaces relevant to the SQRT software product. Table 1 below lists internal interfaces between architecture tiers and/or functions within the software product. Table 2 shows external interfaces between the onboard computer and all components it communicates with, with reference to where in the software product the interfacing will be initialized. See SQT-CDR-DMP-0 for specific data format sent across external interfaces such as the antenna.

Table 1: Software Interfaces

Interface	**Type**	**Commands Exposed & Operations**	**Format**
Command Handler & Data Collection Functions	Management to Capabilities Tier	The command handler (main loop in software) will call the functions handling housekeeping, communications, experiment payload, timekeeping, and OBDH functions as shown in the software product tree.	Functions called during the data collection loop read data from sensors over external interfaces described in Table 2.
Command Handler & Storage SD card	Management & Capabilities Tier	Command handler will open, read from, and close files stored locally on the connected SD card.	A function called after a data collection function writes strings to files in storage.
Triggering Function & Storage SD Card	OBDH & Functions Tier	Pressure, altitude, timestamp data is read from local storage (file path strings input). True/False returned based on analysis of if recent data meets trigger conditions, along with an ASCII flag representing which trigger condition was met.	A Function called during data collection loop reads files in storage.
Command Handler & Calibration Functions	Management to Capabilities Tier	Data is read from Pico storage and calibrated based on sensor type.	Specific functions will be made for each sensor to fit unique calibration needs. Functions called during calibration sections of data collection loop do needed math with collected sensor data and return calibrated values.
Pico & Storage SD Card	OBDH & Functions Tier	Calibrated and formatted lines of data (strings) from Pico onboard computer written to files stored locally on SD card.	Functions called with file path and string to write during storage sections of data collection loop.

Table 2: External (Component) Interfaces

Interface	**Type**	**Commands Exposed & Operations**	**Format**
Pico & Thermal Sensor	I2C	Port sent to device (string), device takes reading, device data returned: Array of floats (sensor array science data), list of floats (calibration values), integer (timestamp).	A function called during data collection loop requests data read from sensor address.
Pico & MS6611	I2C	Address sent to device (string), device takes reading, data returned: Floats (temperature and pressure value), list of floats (calibration constants), integer (timestamp).	A function called during data collection loop requests data from sensor address.
Pico & GPS	UART	Port, NMEA sentence type sent to device (strings), device reads for chosen sequence, device returns sequence of requested type (string) and timestamp (integer).	A function called during data collection loop requests data from sensor address.
Pico & Antenna	UART	Port, Address, Callsign, Data Packets sent to Antenna, Data packets transmitted in 2 packet types (science, telemetry) from the Pico to the antenna, which transmits the packets to ground.	Two functions will be called depending on time during data collection loop to send formatted data packets to antenna.
Pico & DF9GMS	PWM Pin	PWM frequency sent to device (integer), servo motor actuates.	A function is called based on triggering during interruption of data collection loop to send signal to component address.
Pico & DS18B20	OneWire	Port sent to device, device takes reading, device returns float (temperature), timestamp (integer).	A function called during data collection loop requests data read from sensor address.

6 Software Development & Verification Plan

The tasks outline in Table 3 will start in week 6 (Phase 2) and will be completed during Phase 3 to develop and verify the mission software. Associated work package numbers are attached to each task, and ownership of work packages along with a timeline for said work packages are detailed in SQT-CDR-PMP-0. All software will be uploaded and updated in the SQRT GitHub repository during development.

Table 3: Software Development and Verification Tasks

Task	**Associated WP**	**Deliverables**	**Verification Plan**
Preform trade-off analysis for thermal sensor calibration sequence	WP4000	Determine sequence of thermal sensor data capture and calibration based on computing time.	T & A: Test code running on onboard computer with simulated data.
Write/Adapt data collection code for sensors	WP4100	Develop functions to collect and parse data within each sensor class.	T: Each sensor will be tested individually with the code developed to run it, then the system will be tested with components integrated together.
Write robust triggering algorithm	WP4100	The triggering alogrithm used to activate the valve at a set pressure (or other fail-safe conditions)	T: The triggering algorithm will be tested using simulated data. Low pressure, high altitude, and rapid descent conditions cannot be safely created physically in lab, therefore simulated data is needed.
Write data processing code	WP4200	Develop functions to calibrate data within each function class and to analyse data onboard.	T: Functions will be tested alongside data collection functions at a component level followed by a system level. Where possible, data will be collected in lab. Where impossible, simulated data will be created and used.
Write communications code	WP4300	Develop functions to communicate data packets to ground	T: Test communication in lab with the antenna.
Write storage functions	WP4400	Develop functions to store processed/calibrated data on the onboard SD card.	T: Functions will be tested in lab via uploading data to the SD card.
Write command handling software	WP4100	Develop software to call key functions	T: Test overall command handling by running the data collection and transmitting loop with all components integrated.

References

[1] Adafruit. *Adafruit MLX90640 Micropython GitHub*, 2025.

[2] Guido Van Rossom. *PEP8 Style Guide for Python Code*, 2013.